

HelixMemory

HelixMemory

AI एजेंट्स के लिए एक स्मृति मस्तिष्क — चार श्रेष्ठ इंजनों का संलयन।

सारांश

HelixMemory एक Go SDK है जो चार अग्रणी स्मृति प्रणालियों (Mem0, Cognee, Letta, Graphiti) को एक एकीकृत संज्ञानात्मक स्मृति इंजन में समाहित करता है, जो उन्हें समानांतर रूप से खोजता है और परिणामों का संलयन करता है। यह AI अनुप्रयोगों को चार अलग-अलग स्मृति परतों के बजाय एक टिकाऊ, डिडुप्लिकेटेड और पुनः क्रमित स्मृति परत प्रदान करता है।

संक्षिप्त विवरण

HelixMemory एक Go SDK है जो Mem0, Cognee, Letta और Graphiti को AI अनुप्रयोगों के लिए एक एकीकृत संज्ञानात्मक स्मृति इंजन में समाहित करता है। यह लिखने की प्रक्रिया को बुद्धिमानी से निर्देशित करता है, सभी बैकएंड्स को समानांतर रूप से खोजता है और परिणामों को तीन चरणों वाली संग्रहण-डिडुप्लिकेशन-पुनः क्रमन पाइपलाइन के माध्यम से संलयित करता है।

विस्तृत विवरण

HelixMemory, AI अनुप्रयोगों के लिए एक एकीकृत संज्ञानात्मक स्मृति इंजन है, जिसे Go SDK के रूप में प्रस्तुत किया गया है (माइक्रूल `digital.vasic.helixmemory`, Go 1.25+)। इसकी मूल धारणा यह है कि कोई भी एकल स्मृति परियोजना कभी भी हर चीज में श्रेष्ठ नहीं हो सकती — इसलिए स्मृति को खरोच से पुनः लागू करने और एक परियोजना की कमियों को विरासत में लेने के बजाय, यह चार श्रेष्ठ प्रणालियों का संचालन करता है और प्रत्येक को अपनी ताकत के अनुसार काम करने देता है: Mem0 गतिशील तथ्य निष्कर्षण और प्राथमिकता प्रबंधन के लिए, Cognee ECL पाइपलाइनों के माध्यम से निर्मित सिमेंटिक ज्ञान ग्राफ़ के लिए, Letta संपादन योग्य स्मृति ब्लॉक्स और स्लीप-टाइम कंप्यूट के साथ एक स्टेटफुल एजेंट रनटाइम के लिए, और Graphiti एक द्वि-कालिक ज्ञान ग्राफ़ के लिए जो समय के साथ तथ्यों में होने वाले परिवर्तनों पर तर्क करता है।

एक संलयन इंजन ही इन चार स्वतंत्र भंडारों को एक मस्तिष्क में बदलता है। लिखने की प्रक्रिया में, हर आने वाली स्मृति को सामग्री के आधार पर वर्गीकृत किया जाता है और उसे धारण करने के लिए सबसे उपयुक्त बैकएंड पर भेजा जाता है। पढ़ने की प्रक्रिया में, एक क्वेरी

सभी बैकएंड्स पर समानांतर रूप से प्रसारित होती है और कच्चे परिणाम तीन चरणों वाली संलयन पाइपलाइन — संग्रहण, डिडुप्लिकेशन, फिर क्रॉस-सोर्स पुनः क्रमन — में प्रवाहित होते हैं, ताकि कॉलर को चार शोरगुल वाले, अतिव्यापी परिणाम सेटों के बजाय केवल एक स्वच्छ, क्रमित उत्तर मिले। प्रत्येक बैकएंड को सर्किट ब्रेकर से लपेटा गया है ताकि सुचारू गिरावट सुनिश्चित हो सके जब कोई इंजन विफल होता है, तो उसका ब्रेकर ट्रिप हो जाता है और शेष बैकएंड्स सेवा देते रहते हैं, बजाय इसके कि पूरी स्मृति परत ही ध्वस्त हो जाए। चूंकि यह इंजन एक ड्रॉप-इन MemoryStore इंटरफ़ेस को लागू करता है, इसलिए यह एक साधारण स्मृति प्रदाता के सीधे प्रतिस्थापन के रूप में काम करता है — कॉलर को पुनः संरचना करने की आवश्यकता नहीं होती — और Prometheus मेट्रिक्स रूटिंग और संलयन की आंतरिक प्रक्रियाओं को पूर्ण अवलोकन क्षमता प्रदान करते हैं।

HelixMemory को HelixAgent के लिए स्मृति परत के रूप में विकसित किया गया था, जो व्यापक Helix AI समूह का हिस्सा है, और यह परिवार की 'एंटी-ब्लफ़' परीक्षण अनुशासन को स्मृति क्षेत्र में ले आता है: एक इन-प्रोसेस चैलेंजर रनर वास्तविक उत्पादन कोड पथों — रूटिंग, संलयन, ट्रांसलेटर, सर्किट ब्रेकर — का अभ्यास करता है, जबकि एक युग्मित-म्यूटेशन रैपर जानबूझकर इनवेरिएंट्स को उलट देता है ताकि यह साबित हो सके कि जब तक टूटा होता है, तो परीक्षण वास्तव में विफल होते हैं, जिससे एक हरा सूट कुछ मायने रखता है।

हमने इसे क्यों बनाया

सामग्री

AI एजेंटों को दीर्घकालिक, उच्च-गुणवत्ता वाली स्मृति की आवश्यकता होती है, लेकिन पारिस्थितिकी तंत्र बिखरा हुआ है — हर स्मृति परियोजना (Mem0, Cognee, Letta, Graphiti) एक काम में माहिर है तो दूसरे में कमजोर। HelixMemory को HelixAgent के लिए एक ऐसी एकीकृत स्मृति सतह के रूप में बनाया गया है जो उनकी खूबियों को जोड़ती है, बिना किसी एक में बंधने के दबाव के।

यह क्यों गेम-चेंजर है

यह मजबूर चयन को खत्म करता है। चार स्मृति प्रणालियाँ, जो सामान्यतः एक ही स्थान के लिए प्रतिस्पर्धा करती हैं, अब एक ही इंटरफ़ेस के पीछे पूरक बैकएंड बन जाती हैं — जिससे एक एप्लिकेशन को गतिशील तथ्य निष्कर्षण, सिमेंटिक ज्ञान ग्राफ़, स्टेटफुल एजेंट स्मृति और द्वि-कालिक तर्क एक साथ मिलता है, जिसमें डुप्लीकेशन और क्रॉस-सोर्स रीरैंकिंग स्वचालित रूप से संभाली जाती है। जो पहले व्यावहारिक नहीं था, उसे अब “हमें कौन-सी स्मृति इंजन अपनानी चाहिए?” का झूठा द्वंद्व मानने की ज़रूरत नहीं है: HelixMemory आपको एक ही MemoryStore के पीछे उनकी सभी खूबियाँ एक साथ देता है, बिना किसी एक इंजन की कमजोरी को अपनाए या किसी में बंधने के।

क्या है नवीन

- **मल्टी-बैकएंड फ्यूजन** (संग्रह → डिडुप्लीकेट → क्रॉस-सोर्स रीरैंक) जो एक ही रैंकड परिणाम सेट लौटाता है, बजाय इसके कि कॉलर को किसी एक स्टोर से जोड़ दिया जाए।

- **इंटेलिजेंट राइट रूटिंग** जो हर स्मृति को सामग्री के आधार पर वर्गीकृत करती है और उसे उस इंजन तक भेजती है जो उसे संग्रहित करने के लिए सबसे उपयुक्त हो, ताकि सही डेटा सही स्टोर में पहुँचे।
- **ग्रेसफुल डिग्रेडेशन प्रति-बैकएंड सर्किट ब्रेकर** के माध्यम से — एक असफल इंजन को अलग कर दिया जाता है, न कि घातक बनाया जाता है, और बाकी इंजन सेवा देते रहते हैं।
- **स्लीप-टाइम कंप्यूट समेकन** (Letta के माध्यम से) जो निष्क्रिय अवधि के दौरान स्मृति को पुनर्गठित करता है, न कि केवल क्वेरी के समय।
- **एंटी-ब्लफ़ वेरिफिकेशन**: वास्तविक प्रोडक्शन कोड पर चलने वाला चैलेंजर रनर, जिसके साथ एक म्यूटेशन रैपर जुड़ा होता है जो किसी इन्वेरिएंट के उलटने पर असफल होना चाहिए — यह साबित करता है कि टेस्ट गेट एक वास्तविक जाँच है, न कि एक तौटोलॉजी।

सबसे बड़ी तकनीकी चुनौतियाँ और उनके समाधान

- **चार विषम बैकएंड को एक सुसंगत परिणाम सेट में समेटना** — हर इंजन स्मृति को अपने तरीके से लौटाता है, और उन्हें साधारण तरीके से मिलाने पर डुप्लीकेट और असंगत रैंकिंग पैदा होती है। इसका समाधान एक टाइप्ड फ्यूज़न इंजन से किया गया जो सभी स्रोतों से डेटा एकत्र करता है, ओवरलैप को हटाता है, और सब कुछ एक समान आधार पर रैंक करता है, जिसमें फ्यूज़्ड-काउंट इन्वेरिएंट को टेस्ट में सत्यापित किया जाता है ताकि मर्ज के दौरान चुपचाप परिणाम न खोएँ या दोहराए न जाएँ।
- **बैकएंड के डाउन होने पर भी सेवा जारी रखना** — एक **недоступный** स्मृति इंजन पूरे लेयर को रोक नहीं सकता। इसका समाधान प्रति-बैकएंड सर्किट ब्रेकर से किया गया जो क्लोज्ड → ओपन (विफलता सीमा के बाद) → हाफ-ओपन (टाइमआउट के बाद) की स्थिति में काम करता है, अस्वस्थ बैकएंड को अलग करता है और स्वस्थ बैकएंड से सेवा जारी रखता है जब तक वह ठीक नहीं हो जाता।
- **स्मृति तर्क के वास्तव में काम करने का प्रमाण, न कि केवल कंपाइल होने का** — एक हरा-भरा टेस्ट सूट बेकार है अगर टेस्ट असफल नहीं हो सकते। इसका समाधान एक इन-प्रोसेस चैलेंजर रनर से किया गया जो वास्तविक प्रोडक्शन कोड (रूटिंग, फ्यूज़न, ट्रांसलेटर, सर्किट ब्रेकर) को चलाता है और एक म्यूटेशन रैपर के साथ जोड़ा जाता है जो इन्वेरिएंट को उलट देता है और मांग करता है कि टेस्ट लाल हो जाएँ, ताकि यह साबित हो सके कि गेट एक वास्तविक जाँच है, न कि तौटोलॉजी।

सामग्री

तकनीकी संरचना

- **Go (1.25+)** — एकल SDK और रनटाइम; इसे इसलिए चुना गया क्योंकि चार बैकएंड्स में समानांतर रीड फैन-आउट एक समवर्ती समस्या है, और Go के गो-रूटीन इसे सस्ता बनाते हैं, जबकि इसके इंटरफ़ेस प्रकार पूरे सिस्टम को एक स्वच्छ सीम (MemoryStore) प्रदान करते हैं, जिस पर कॉलर निर्भर रह सकते हैं।
- **Mem0** — गतिशील तथ्य-निष्कर्षण और प्राथमिकता-प्रबंधन बैकएंड; इसका उपयोग “उपयोगकर्ता वास्तव में क्या पसंद करता है / कौन से तथ्य सामने आए हैं” के स्मृति खंड के लिए किया जाता है।
- **Cognee** — ECL पाइपलाइनों पर आधारित सिमेंटिक ज्ञान-ग्राफ बैकएंड; इसका उपयोग संरचित, संबंधित ज्ञान को संग्रहीत करने के लिए किया जाता है, न कि केवल सपाट तथ्यों के लिए।

- **Letta** — संपादन योग्य स्मृति ब्लॉक्स और स्लीप-टाइम कंप्यूट के साथ स्टेटफुल एजेंट-रनटाइम बैकएंड; इसका उपयोग वहाँ किया जाता है जहाँ स्मृति को जीवंत एजेंट स्थिति के रूप में बने रहना चाहिए और निष्क्रिय अवधि के दौरान समेकित किया जाना चाहिए।
- **Graphiti** — द्वि-समयिक ज्ञान-ग्राफ बैकएंड; इसका उपयोग तथ्यों और संबंधों के समय के साथ बदलने पर तर्क करने के लिए किया जाता है, न कि केवल उनके वर्तमान मूल्य के लिए।
- **PostgreSQL + Neo4j + Redis** — वास्तविक डेटा स्टोर जिनके विरुद्ध बैकएंड चलते हैं, जिन्हें make infra-start के माध्यम से वास्तविक एकीकरण परीक्षण के लिए स्थापित किया गया है ताकि सूट लाइव इंफ्रास्ट्रक्चर का उपयोग करे न कि मॉक का।
- **Prometheus** — मेट्रिक्स और ऑब्जर्वेबिलिटी को फ्यूज़न पाइपलाइन के माध्यम से जोड़ा गया है, ताकि रूटिंग और फ्यूज़न व्यवहार को उत्पादन में मापा जा सके, न कि एक ब्लैक बॉक्स के रूप में।
- **अंतर्राष्ट्रीयकरण अनुवादक सीम** — एक नामित (helixmemory_) स्ट्रिंग सतह को बनाए रखा गया है ताकि भविष्य में किसी भी उपयोगकर्ता-सामना वाले परत को मुख्य प्रणाली में बदलाव किए बिना स्थानीयकृत किया जा सके।

स्थिति और ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** बीटा। कार्यशील SDK; HelixAgent के लिए स्मृति परत के रूप में निर्मित।
- **लाइसेंस:** अज्ञात। GitHub API के माध्यम से कोई LICENSE नहीं पाई गई — अप्रमाणित / घोषित नहीं।
- **प्रदर्शन नाम** “HelixMemory” रिपॉजिटरी memory से मेल खाता है। README में उल्लिखित सटीकता के आँकड़े अपस्ट्रीम विक्रेताओं के दावे हैं, HelixMemory के माप नहीं, और यहाँ उन्हें छोड़ दिया गया है।

प्राथमिकता स्तर: Helix-प्राथमिक।